Passive Piezo Sounder

Audible cues without blocking the loop

Own the timer, pass time explicitly, and make silence the safe state.

Lesson	005 — component	**Time**	80–110 minutes
Requires	Digital output and explicit time	**Board**	Arduino Mega 2560
Output	Passive piezo on D6	**Status**	host verified; hardware experimental
Safety	E1: identified logic-level passive piezo only	**Power**	Mega USB only

1 Why sound is a resource problem

A tone looks like a simple output, but it couples a pin to a hardware timer. ADK's `PiezoSounder` claims both resources before it touches the circuit. If either is busy, initialization fails predictably. Once initialized, `play()` starts a square wave and returns immediately. The main loop remains free to scan buttons, update LEDs, and enforce deadlines.

Learning targets

By the end, you can:

- distinguish a passive piezo transducer from an active buzzer or speaker;
- wire a modest passive piezo load to Mega D6 with a series resistor;
- explain why tone owns D6 and Timer2;
- schedule and stop a tone using explicit `TimePoint` values;
- predict pin and timer claim conflicts before hardware changes;
- verify duration, replacement, shutdown, and rollover deterministically;
- show that stop, shutdown, and destruction return the endpoint to silence.

2 Part boundary and safety

Safety checkpoint. Use only a small *passive piezo transducer* documented for logic-level excitation. Do not substitute an 8 Ω speaker, magnetic sounder, siren, active buzzer module, amplifier, or unknown part. Disconnect USB and every other power source before changing wiring. Begin with short tones at a comfortable distance; stop if sound is painful, startling, or causes discomfort.

Identify the part before building:

- A passive piezo needs an alternating waveform; steady DC produces at most a click. Kits may label it "passive buzzer".
- An active buzzer contains an oscillator and sounds from steady DC. It is a different component and is not used here.
- A dynamic speaker is a low-impedance inductive load. A Mega pin must not drive it directly.

Transistor/speaker boundary. A speaker, high-current sounder, or louder transducer requires a separately designed driver: a suitable transistor or amplifier, current limiting, supply analysis, common-ground or isolation decision, and inductive protection where the load requires it. That driver is outside this lesson. A transistor is not permission to guess a load's ratings.

Required: Mega 2560, USB data cable, breadboard, one passive piezo transducer, one 100 Ω series resistor, and jumper wires. Use the transducer datasheet when its permitted voltage or wiring differs. Record the manufacturer's part number, rated voltage, and capacitance or drive limits before energizing it. If the kit part cannot be identified well enough to establish those limits, keep the exercise host-only.

3 Predict before wiring

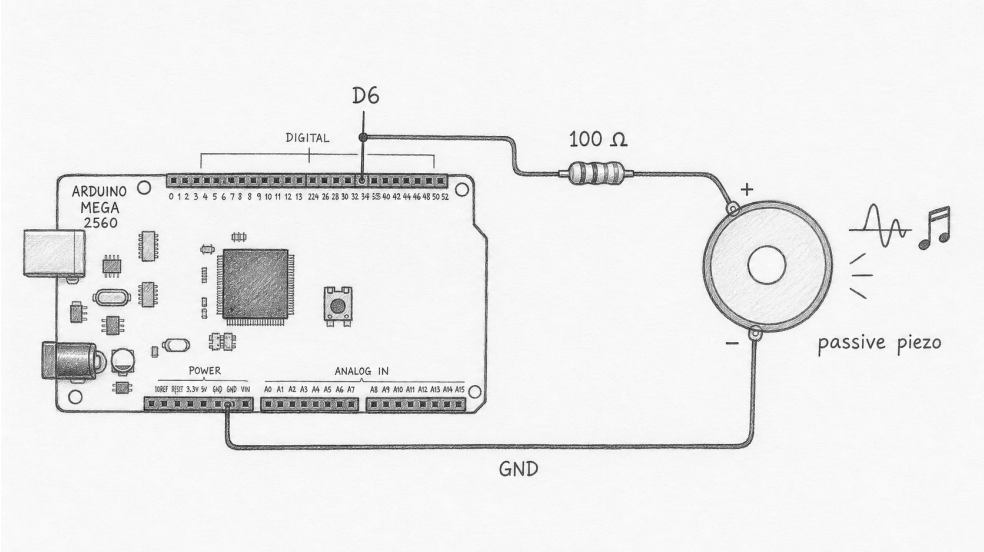
Action	Expected software state	Expected circuit state
Construct only	not initialized, not sounding	D6 untouched
Initialize	initialized, not sounding	D6 passive; Timer2 owned
Play 440 Hz for 250 ms	sounding at 440 Hz	square wave drives piezo
Update before deadline	still sounding	waveform continues
Update at/after deadline	not sounding	tone stopped; D6 restored to input
Shutdown or destruction	not initialized, not sounding	silence; claims released

Conflict prediction: If another component owns Timer2, which status should `initialize()` return? _____

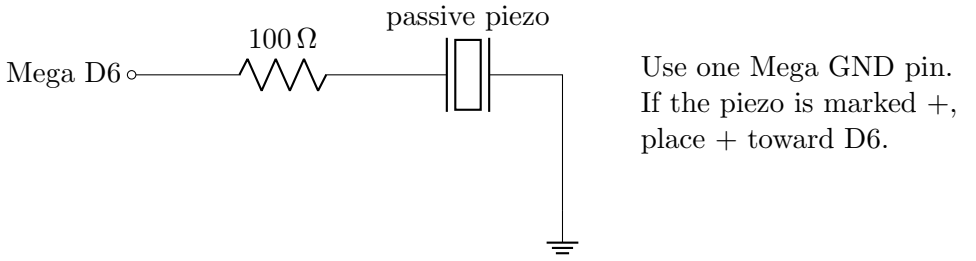
Should D6 change mode? _____ Why?

_____

4 Exact Mega 2560 circuit



Orientation only. This drawing identifies the intended parts and signal path, but pin spacing and board geometry are illustrative. The exact schematic and connection table below are authoritative. Trace D6, 100 Ω, passive piezo, and Mega GND before applying power.



From	Through	To
Mega D6	100 Ω resistor	passive piezo signal / + terminal
piezo return / − terminal	jumper	Mega GND

The resistor limits edge current into the piezo's capacitance and makes the exercise conservative; it does not make an arbitrary load safe. Do not connect a separate 5 V wire. Keep leads short. With power disconnected, verify no continuity from 5 V to GND and trace D6 through the resistor and piezo to GND.

5 Use the exact interface

```
#include <Adk.h>

constexpr adk::PinId                    SounderPin = 6;
constexpr adk::PiezoSounder::Frequency CueHz      = 440;
const adk::Duration                     CueLength  (250);

adk::Runtime      runtime;
adk::PiezoSounder sounder (runtime.resources (), SounderPin);

void setup ()
{
    const adk::Status status = sounder.initialize ();

    if (!status.ok ())
    {
        return;
    }

    const adk::TimePoint now (millis ());
    sounder.play (CueHz, CueLength, now);
}

void loop ()
{
    const adk::TimePoint now (millis ());
    sounder.update (now);
}
```

`play(frequency, duration, now)` accepts 31–20,000 Hz and a duration of 1–60,000 ms. Zero duration, an out-of-range frequency, or an excessive duration returns a status whose `error()` is `StatusCode::InvalidArgument` without starting a new tone. `play()` before initialization reports `error()` as `StatusCode::NotInitialized`.

The API does not call `delay()`. It stores the supplied start time and duration. Each `update(now)` compares the caller's time with the deadline. The caller therefore controls replay, test time, and loop cadence.

5.1 Lifecycle and replacement

Construction only stores the registry and pin. Successful initialization claims D6 first, then Timer2. If Timer2 is unavailable, the D6 claim is rolled back. Repeated successful initialization is harmless.

A successful `play()` while already sounding replaces the frequency, duration, and start time. It does not queue a tone. `stop()` is idempotent: it calls the platform stop operation only while sounding, clears the audible state, and returns D6 to input mode. `shutdown()` stops first, releases Timer2 and D6, and is also idempotent. The destructor calls shutdown, so cleanup remains correct in an exception-using host environment even though ADK throws no exceptions internally.

6 Timer2 ownership

On the Mega 2560 target, this implementation models Arduino tone generation as exclusive ownership of timer index 2. Timer2 is also associated with hardware PWM channels on Mega pins 9 and 10. Board-aware PWM code and PiezoSounder must therefore agree through the same `ResourceRegistry`; incompatible use fails instead of silently changing frequency or breaking a tone.

Existing owner	Sounder initialization	Required evidence
D6 pin owner	`error(): ResourceBusy`	no pin mode or tone call
Timer2 owner	`error(): ResourceBusy`	D6 claim rolled back; no hardware call
Unrelated pin/timer owner	`ok(): true`	D6 and Timer2 claimed
Same initialized sounder	`ok(): true`	no repeated hardware work

> **Safety checkpoint.** A registry can prevent conflicts only when every participating component uses that registry and the board map describes the shared timer correctly. Direct calls to Arduino `tone()`, `analogWrite()`, or third-party timer libraries bypass ADK ownership. Do not mix them in this lesson.

7 Deterministic host tests

The implementation is host verified with fake Arduino tone operations. Add or review tests that drive exact time values; never sleep in a correctness test.

```
fake::reset ();
adk::ResourceRegistry resources;
adk::PiezoSounder     sounder (resources, 6);

require (sounder.initialize ().ok ());
require (sounder.play
         (
             440,
             adk::Duration  (250),
             adk::TimePoint (1000)
         ).ok ());

sounder.update (adk::TimePoint (1249));
require        (sounder.sounding ());
```

```
sounder.update (adk::TimePoint (1250));
require        (!sounder.sounding ());
require        (sounder.frequency () == 0);
```

Required deterministic cases:

1. construction makes no Arduino call;
2. initialization claims pin and Timer2 before touching hardware;
3. pin conflict and timer conflict return `ResourceBusy`;
4. timer-claim failure releases the temporary pin claim;
5. play before initialization returns `NotInitialized`;
6. boundary frequencies 31 and 20,000 Hz succeed;
7. frequency 30 or 20,001 Hz, zero duration, and 60,001 ms fail;
8. update one millisecond before the deadline preserves the tone;
9. update exactly at and after the deadline stops it;
10. a second play replaces the first tone and restarts its deadline;
11. explicit `stop()`, repeated stop, shutdown, repeated shutdown, and destruction produce the minimal safe trace;
12. elapsed-time arithmetic works across 32-bit millisecond rollover.

Rollover vector. Start at `TimePoint(0xffffffff0)`, play for 32 ms, update at `TimePoint(0x0000000f)` and then `TimePoint(0x00000010)`. The first update is one millisecond early; the second reaches the deadline.

8 Hardware investigation

Start with 440 Hz for 100 ms. Keep the piezo away from your ear. Increase to 250 ms only after confirming comfortable output. Run three trials.

Trial	Frequency	Requested duration	Observed result	Loop remained responsive?
1	440 Hz	100 ms	_____	_____
2	660 Hz	100 ms	_____	_____
3	880 Hz	250 ms	_____	_____

While a tone runs, update the Lesson 001 diagnostic LED at a visible cadence or scan the Lesson 002 input. Continued response is evidence that tone duration is nonblocking. It is not evidence that every possible frequency is equally loud or safe; piezo response varies strongly with resonance.

8.1 Safe fault exercises

Remove power before each wiring change, change one condition, then restore the exact schematic:

- disconnect D6: software reports sounding, but the transducer is silent;
- disconnect GND: same symptom, different open path;
- request 30 Hz: expect `InvalidArgument` and silence;
- pre-claim D6 or Timer2 in a host test: expect `ResourceBusy` before hardware work;
- call `shutdown()` during a long tone: expect immediate silence.

Do not short D6, omit the resistor as a "fault," substitute a speaker, or probe the powered circuit with a meter in current mode.

9 Mega 2560 acceptance

Board: _____ port: _____
Toolchain: _____ date: _____

Piezo manufacturer / part: _____ rated voltage: _____ capacitance: _____

☐ Part is identified as a passive piezo with suitable rating.
☐ Exact D6–100 Ω–piezo–GND path is verified unpowered.
☐ Clean Mega build succeeds; flash and SRAM sizes are recorded.
☐ Initialization returns a status whose `ok()` is true.
☐ Three requested cues sound and end without `delay()`.
☐ A simultaneous LED or input task remains responsive.
☐ Invalid frequency returns `InvalidArgument` and remains silent.
☐ Explicit stop ends a tone immediately.
☐ Shutdown leaves D6 passive and releases both claims.
☐ USB is disconnected before dismantling the circuit.

Flash: _____ bytes SRAM: _____ bytes Longest observed cue: _____ ms

10 Explain and extend

Claim: Does the evidence support nonblocking, deadline-controlled sound?

_____

Evidence: Cite a tone boundary, a concurrently updated component, and a safe shutdown observation.

_____

_____

Reasoning: Explain how explicit time and exclusive Timer2 ownership make the behavior predictable.

_____

Limitation: What remains hardware experimental after host tests pass?

_____

Next, map a few named cues to fixed frequencies without hiding `Status` results. Keep sequences deterministic: a cue table supplies data; the loop supplies time. Lesson 006 combines output, input, PWM color, and audible feedback in Simon.

11 References

The HTML lesson carries current commands, source links, API signatures, errata, and test navigation. This PDF emphasizes bench procedure and evidence.

- Arduino Mega 2560 Rev3 documentation and pinout
- Arduino tone melody example
- Arduino tone reference
- Microchip ATmega2560 datasheet
- ADK documentation